Journal — Hardik Pajiala (1024030119)

Role: Person A — Frontend & UX (React) **Project:** FoodBridge — a surplus food redistribution matcher

Week 4 (27 October 2026)

What I did

Took over Person C's location work. Ayush is not completing it, so I picked up the geocoding integration. Reading it first turned up the real problem: the code existed but nothing could reach it. `donationService` did call the geocoding service, but the donor form always sent coordinates, so the address branch was unreachable. `authService` never imported it at all — it accepted an address, stored it, and then refused any organisation that could not supply its own latitude and longitude. The `geocoding_cache` table was empty, as it had been since we created it.

Added `POST /api/geocode`. It resolves an address through the cache first and the provider only on a miss, and reports which of the two answered, so the client can show where a place actually landed.

Made registration resolve an address. An organisation can now register by typing where it is, instead of being required to know its coordinates. Explicit coordinates still win when they are given.

Built one location control for both forms. Type an address, press Find, and the resolved point is shown before anything is submitted. Editing the text clears the previous result, so a stale point can never be submitted under a new address. The five Patiala presets became quick-fill that still go through the server rather than carrying hardcoded coordinates.

Caught a bug of my own by walking the interface. I had put `/geocode` behind authentication, which is wrong: an organisation has to locate its address while it is still registering, before it has a token. Registering was therefore impossible. The endpoint is now open, with a per-caller rate limit instead, because Nominatim's usage policy applies to our whole application — one abusive caller could get the project blocked.

Fixed a flaky check I introduced last week. I had promoted the match-inbox conformance check to required, but whether our organisation wins a given donation depends on the scores, so an empty inbox is correct behaviour. It now fails only when an offer exists and is missing its joined donation.

Verified: 73 backend tests pass (64 last week), 30 matching-service tests, 26 contract checks with no failures, lint and build clean. Proved live that the first lookup is `GEOCODED` and an identical second one is `CACHE`, and posted a donation through the interface at a typed address — Rajindra Hospital, Patiala, resolved to 30.3292, 76.3839. The cache now holds real rows for the first time.

What I learned

An endpoint's access rules have to match *when* in the user's journey it is needed, not just what it does. Geocoding looks like something a signed-in user does, so putting it behind a token felt obviously right, and it silently made registration impossible. No test caught it because every test already had a token.

Opening that endpoint is not free either. The rate limit is there because the provider's limit applies to the application as a whole rather than per caller, so an open proxy to someone else's service is a liability we would carry.

This is last week's lesson from the other side. Then, a field existed at every layer and was dead because the form never sent it. This time the service was correct and fully tested, and was dead because the form always sent coordinates instead. Both times the code was right and the path to it was not.

Blocked / needs the team

- The frontend is still not in the main repository.
- The rest of Person C's scope needs an owner on paper, not just in practice.
- The rate limiter keeps its counts in memory, so the window is per process. Fine for one instance; a multi-instance deployment would need shared state.

Next week

Get the frontend onto the main repository, and test the address flow on a real handset rather than an emulated viewport — still outstanding.